

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №5

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Застосування нейро-нечітких мереж для прогнозування ризиків (керування виробничим процесом)”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Основною метою цієї роботи є глибоке ознайомлення з теоретичними основами та практичним застосуванням нечіткої логіки та гібридних нейро-нечітких систем (ANFIS). Навчитись виконувати ключові етапи роботи з нечіткими системами, як-от фазифікація та дефазифікація даних, а також реалізувати повноцінну модель для прогнозування ризиків у виробничому процесі, використовуючи сучасні бібліотеки Python, такі як scikit-fuzzy та PyTorch. Фінальним етапом є проведення експериментальних досліджень для оцінки ефективності розробленої моделі та формулювання обґрунтованих висновків.

Індивідуальне завдання

- Сформувати нечіткі множини для вхідних змінних (температура, тиск) та вихідної змінної «ризик».
- Побудувати базу правил типу IF-THEN, яка є ядром нечіткої системи.
- Реалізувати ANFIS-модель та порівняти її точність і гнучкість із класичною нечіткою системою.
- Провести навчання та оцінку моделей, візуалізувати результати за допомогою графіків (крива втрат, порівняння прогнозів) та розрахувати метрики ефективності (RMSE, MAE, R).

Оберіть завдання відповідно до вашого номера у списку групи. Виконайте його, детально опишіть хід роботи, додайте необхідні графіки та таблиці, а також зробіть висновок за результатами.

№	Batch size	Epochs	Оптимізатор	Додаткове завдання
3	128	150	RMSprop	Візуалізувати приклади прогнозів.

Рекомендації щодо налаштувань

- Починайте з 5–8 правил; збільшуйте лише за потреби.
- Тип ФН: gauss для гладких меж, trimf для простоти/інтерпретації.
- Оптимізатор: Adam (швидка збіжність), RMSprop (стабільний на «шумних» даних), SGD (для контрольованих експериментів).
- Регуляризація: weight_decay, рання зупинка за val-втратою.
- Для класифікації ризику – встановіть поріг 0.5 або підберіть його за максимальним F1/Youden J з ROC.

Хід роботи

1. Постановка задачі

На цій лабораторній потрібно було провести дослідження та практичне застосування гібридних нейро-нечітких систем (ANFIS) для вирішення задачі прогнозування.

Конкретна задача полягала в тому, щоб розробити модель, здатну прогнозувати рівень ризику (числове значення від 0 до 1) у виробничому процесі на основі трьох вхідних параметрів:

- Температура (Т, °C)
- Тиск (Р, Бар)
- Вологість (Н, %)

Для цього я використовував синтетичний набір даних, що імітує складну нелінійну залежність між цими параметрами та рівнем ризику. Також я мав порівняти точність розробленої ANFIS-моделі з класичною нечіткою системою (FIS) типу Мамдані.

Індивідуальне завдання для мого 3-го варіанту полягало в налаштуванні ANFIS-моделі з наступними гіперпараметрами:

- Batch size: 128
- Epochs: 150
- Оптимізатор: RMSprop
- Додаткове завдання: Візуалізувати приклади прогнозів.

2. Програмна реалізація

Для виконання роботи я використовував середовище Google Colab та мову програмування Python з бібліотеками PyTorch, scikit-fuzzy, Numpy, Scikit-learn та Matplotlib.

Мій код складався з наступних розділів:

1. Генерація та підготовка даних:

Я згенерував 1200 зразків даних, де входи (Температура, Тиск, Вологість) були випадковими величинами, а вихід (Ризик) розраховувався за складною формулою з додаванням шуму. Потім об'єднав вхідні дані в єдину матрицю X та нормалізував її за допомогою MinMaxScaler до діапазону [0, 1]. Це є важливою вимогою для стабільного навчання нейронних мереж. І на кінець розділив дані на навчальну (80%) та тестову (20%) вибірки (X_train, X_test, y_train, y_test).

```
Розмір навчальної вибірки: 960
Розмір тестової вибірки: 240
```

```
!pip install -U scikit-fuzzy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

```

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
import os
%matplotlib inline
np.random.seed(42)
torch.manual_seed(42)

n_samples = 1200
temp = np.random.uniform(20, 100, n_samples) # Температура, °C
press = np.random.uniform(1, 10, n_samples) # Тиск, Бар
humid = np.random.uniform(10, 90, n_samples) # Вологість, %
risk = (0.5 * (temp / 100) +
        0.3 * (1 - press / 10) +
        0.2 * (humid / 100) +
        0.1 * np.sin(temp / 10) +
        np.random.normal(0, 0.03, n_samples))
risk = np.clip(risk, 0, 1)
# Нормалізація даних до діапазону [0, 1]
X = np.vstack([temp, press, humid]).T
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
# Розбиття на навчальну та тестову вибірки
X_train, X_test, y_train, y_test = train_test_split(
    X_normalized, risk, test_size=0.2, random_state=42
)
print(f"Розмір навчальної вибірки: {X_train.shape[0]}")
print(f"Розмір тестової вибірки: {X_test.shape[0]}")

```

2. Реалізація класичної FIS (для порівняння):

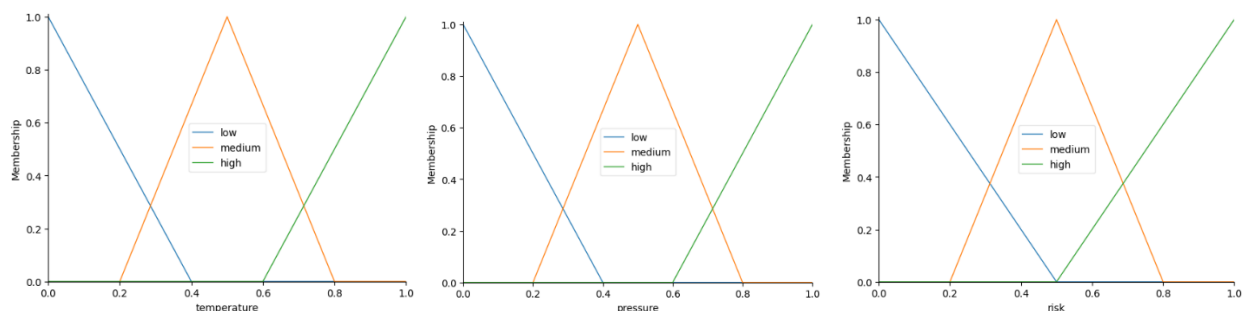
Я створив базову нечітку систему (FIS) типу Мамдані за допомогою scikit-fuzzy. Визначив функції належності (трикутні trimf) для двох входів ('temperature', 'pressure') та виходу ('risk') з трьома термами: 'low', 'medium', 'high'. Вручну визначив 4 простих правила (IF-THEN) і прогнав тестову вибірку через цей симулятор та розрахував метрики якості (RMSE, MAE, R2), щоб мати базовий рівень для порівняння.

Функції належності для класичної системи:

```

--- Метрики Класичної Нечіткої Системи (Mamdani) ---
RMSE: 0.0280
MAE: 0.1276
R2: 0.1039

```



```

# Визначення універсумів та нечітких змінних
t_norm = ctrl.Antecedent(np.arange(0, 1.01, 0.01), 'temperature')
p_norm = ctrl.Antecedent(np.arange(0, 1.01, 0.01), 'pressure')
risk_out = ctrl.Consequent(np.arange(0, 1.01, 0.01), 'risk')
# Визначення функцій належності (трикутні)
t_norm['low'] = fuzz.trimf(t_norm.universe, [0, 0, 0.4])
t_norm['medium'] = fuzz.trimf(t_norm.universe, [0.2, 0.5, 0.8])
t_norm['high'] = fuzz.trimf(t_norm.universe, [0.6, 1.0, 1.0])

p_norm['low'] = fuzz.trimf(p_norm.universe, [0, 0, 0.4])
p_norm['medium'] = fuzz.trimf(p_norm.universe, [0.2, 0.5, 0.8])
p_norm['high'] = fuzz.trimf(p_norm.universe, [0.6, 1.0, 1.0])

risk_out['low'] = fuzz.trimf(risk_out.universe, [0, 0, 0.5])
risk_out['medium'] = fuzz.trimf(risk_out.universe, [0.2, 0.5, 0.8])
risk_out['high'] = fuzz.trimf(risk_out.universe, [0.5, 1, 1])
# Візуалізація функцій належності
print("Функції належності для класичної системи:")
t_norm.view()
p_norm.view()
risk_out.view()
# Формулювання бази правил
rule1 = ctrl.Rule(t_norm['high'] | p_norm['low'], risk_out['high'])
rule2 = ctrl.Rule(t_norm['medium'] & p_norm['medium'], risk_out['medium'])
rule3 = ctrl.Rule(t_norm['low'] & p_norm['high'], risk_out['low'])
rule4 = ctrl.Rule(t_norm['high'] & p_norm['high'], risk_out['medium'])

# Створення системи керування
risk_ctrl_system = ctrl.ControlSystem([rule1, rule2, rule3, rule4])
risk_simulator = ctrl.ControlSystemSimulation(risk_ctrl_system)
# Приклад розрахунку для одного значення
y_pred_fis = []
for i in range(X_test.shape[0]):
    risk_simulator.input['temperature'] = X_test[i, 0] # Вхід 0 - temp
    risk_simulator.input['pressure'] = X_test[i, 1]    # Вхід 1 - press
    # Вхід 2 (humid) ігнорується цією простою моделлю
    try:
        risk_simulator.compute()
        y_pred_fis.append(risk_simulator.output['risk'])
    except:
        y_pred_fis.append(0.5) # Якщо правило не спрацювало, ставимо середнє

y_pred_fis = np.array(y_pred_fis)

print("\n--- Метрики Класичної Нечіткої Системи (Mamdani) ---")
print(f"RMSE: {mean_squared_error(y_test, y_pred_fis):.4f}")
print(f"MAE: {mean_absolute_error(y_test, y_pred_fis):.4f}")
print(f"R2: {r2_score(y_test, y_pred_fis):.4f}")

```

3. Реалізація ANFIS-моделі на PyTorch:

Я визначив архітектуру моделі як клас SimpleANFIS, що успадковує nn.Module. Ця архітектура імітує 5 шарів ANFIS:

- Шар 1 (Фазифікація) та Шар 2 (Сила правила) реалізовані через `nn.Linear (self.premise)` та гаусоподібну функцію активації (`torch.exp(-0.5 * ...)`). Параметри центрів та ширин (`self.scale`) є тими, що навчаються.
- Шар 3 (Нормалізація) реалізований через ділення активацій на їх суму.
- Шар 4-5 (Дефазифікація та Вихід) реалізовані як модель Сугено 0-го порядку за допомогою `nn.Linear (self.consequent)`.

На виході моделі я застосував `torch.sigmoid`, щоб гарантувати, що прогноз ризику завжди буде в діапазоні `[0, 1]`.

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

batch_size = 128

train_dataset = TensorDataset(X_train_t, y_train_t)
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size,
                           shuffle=True)

test_dataset = TensorDataset(X_test_t, y_test_t)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
                          shuffle=False)

class SimpleANFIS(nn.Module):
    def __init__(self, n_inputs=3, n_rules=8):
        super().__init__()
        self.n_inputs = n_inputs
        self.n_rules = n_rules
        self.premise = nn.Linear(n_inputs, n_rules, bias=True)
        self.scale = nn.Parameter(torch.rand(n_rules) * 0.5 + 0.5)
        self.consequent = nn.Linear(n_rules, 1, bias=True)

    def forward(self, x):
        a = self.premise(x)
        scaled_dist = a / (1e-6 + self.scale)
        activations = torch.exp(-0.5 * scaled_dist**2)
        sum_activations = activations.sum(dim=1, keepdim=True)
        weights = activations / (1e-6 + sum_activations)
        y = self.consequent(weights)

        return torch.sigmoid(y) # Використовуємо sigmoid
```

4. Навчання моделі:

Я перетворив дані у `TensorDataset` та `DataLoader` з `batch_size=128`, ініціалізував модель з 10-ма нечіткими правилами, і встановив функцію втрат `nn.MSELoss`. В якості оптимізатора було обрано `torch.optim.RMSprop` з `lr=0.01`.

І на кінець запустив цикл навчання на 150 епох. На кожній епосі я проводив повний прохід по навчальному DataLoader (mini-batch) та розраховував втрати на навчальній (train) та тестовій (val) вибірках.

Початок навчання ANFIS моделі:

Входи: 3, Правила: 10

Оптимізатор: RMSprop

Епохи: 150

Batch Size: 128

Епоха [15/150], Train Loss: 0.00293, Val Loss: 0.00268

Епоха [30/150], Train Loss: 0.00152, Val Loss: 0.00178

Епоха [45/150], Train Loss: 0.00138, Val Loss: 0.00138

Епоха [60/150], Train Loss: 0.00146, Val Loss: 0.00143

Епоха [75/150], Train Loss: 0.00124, Val Loss: 0.00135

Епоха [90/150], Train Loss: 0.00119, Val Loss: 0.00120

Епоха [105/150], Train Loss: 0.00117, Val Loss: 0.00110

Епоха [120/150], Train Loss: 0.00112, Val Loss: 0.00112

Епоха [135/150], Train Loss: 0.00120, Val Loss: 0.00114

Епоха [150/150], Train Loss: 0.00121, Val Loss: 0.00175

Навчання завершено.

```
n_inputs = X_train.shape[1]
```

```
n_rules = 10
```

```
model_anfis = SimpleANFIS(n_inputs=n_inputs, n_rules=n_rules)
```

```
learning_rate = 0.01
```

```
epochs = 150
```

```
optimizer = optim.RMSprop(model_anfis.parameters(), lr=learning_rate)
```

```
loss_fn = nn.MSELoss()
```

```
print(f"Початок навчання ANFIS моделі:")
```

```
print(f" Входи: {n_inputs}, Правила: {n_rules}")
```

```
print(f" Оптимізатор: RMSprop")
```

```
print(f" Епохи: {epochs}")
```

```
print(f" Batch Size: {batch_size}")
```

```
train_losses = []
```

```
val_losses = []
```

```
for epoch in range(epochs):
```

```
    model_anfis.train()
```

```
    batch_train_losses = []
```

```
    for x_batch, y_batch in train_loader:
```

```
        y_pred = model_anfis(x_batch)
```

```
        loss = loss_fn(y_pred, y_batch)
```

```
        optimizer.zero_grad()
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        batch_train_losses.append(loss.item())
```

```
    train_losses.append(np.mean(batch_train_losses))
```

```
    model_anfis.eval()
```

```
    batch_val_losses = []
```

```

with torch.no_grad():
    for x_val, y_val in test_loader:
        y_val_pred = model_anfis(x_val)
        val_loss = loss_fn(y_val_pred, y_val)
        batch_val_losses.append(val_loss.item())

val_losses.append(np.mean(batch_val_losses))

if (epoch + 1) % 15 == 0:
    print(f"Епоха [{epoch+1}/{epochs}], Train Loss: {train_losses[-1]:.5f}, Val Loss: {val_losses[-1]:.5f}")

print("Навчання завершено.")

```

3. Результати експериментів

Після завершення процесу навчання моделі я провів оцінку її ефективності та візуалізував результати.

Я отримав наступні метрики на тестовій вибірці:

```

--- Метрики якості навченої моделі ANFIS ---
RMSE: 0.0018
MAE: 0.0328
R2: 0.9437

```

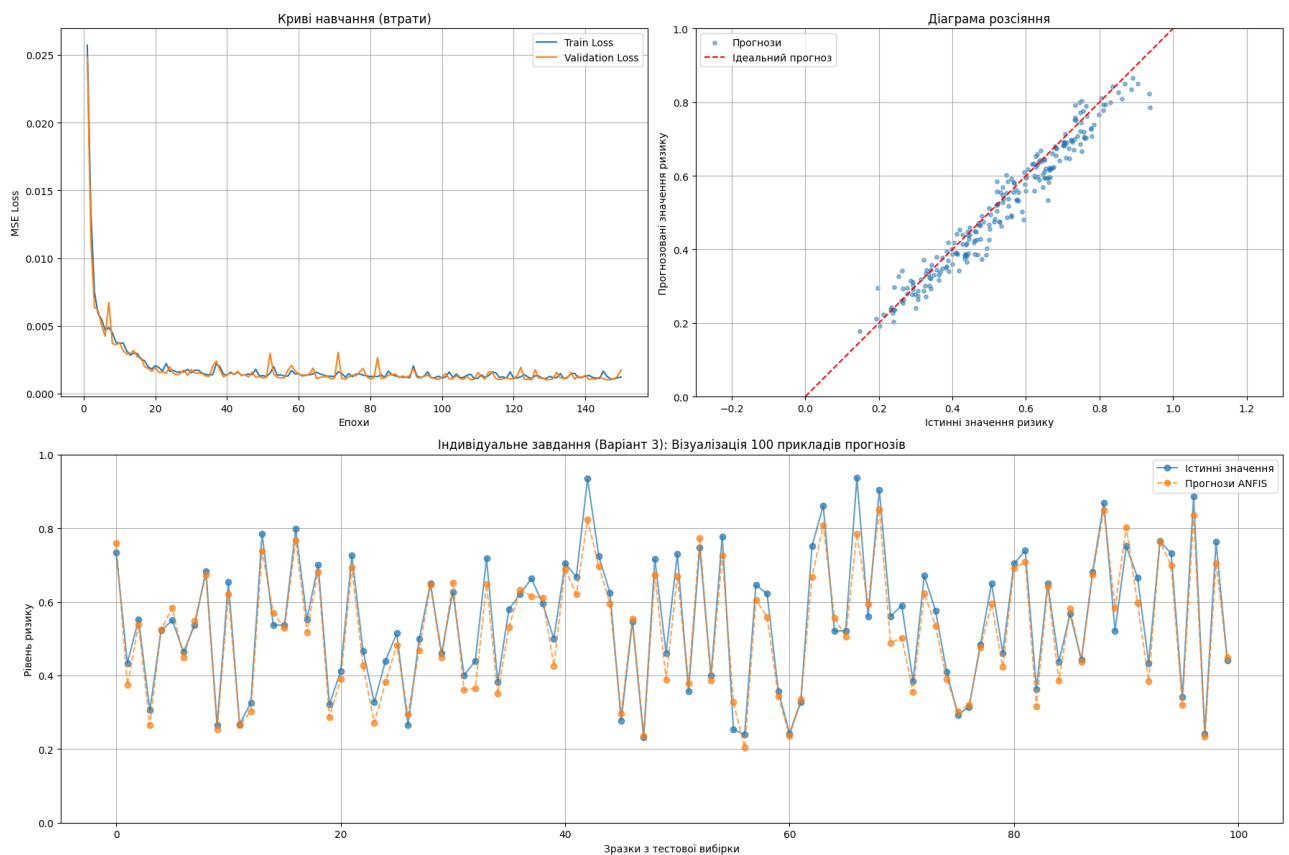
Модель	RMSE	MAE	R ² (R-squared)
Класична FIS (Mamdani)	0.0280	0.1276	0.1039
ANFIS	0.0018	0.0328	0.9437

Як видно з таблиці, навчена ANFIS-модель показує значно кращі результати (нижчі RMSE/MAE та вищий R2), ніж класична система з "ручними" правилами. Це демонструє здатність гібридної мережі адаптуватися до даних та вивчати складні залежності.

Візуалізація результатів:

1. Криві навчання: Я побудував графік втрат (MSE) на навчальній та валідаційній вибірках протягом 150 епох. Графік показує, що модель швидко навчається (втрати знижуються). Втрати на валідації (Val Loss) також знижуються і залишаються близькими до навчальних, що свідчить про відсутність значного перенавчання.
2. Діаграма розсіяння: Цей графік порівнює істинні значення ризику з прогнозованими. Точки збираються щільно навколо діагональної лінії "ідеального прогнозу", що підтверджує високу кореляцію та точність моделі.
3. Візуалізація прогнозів: Я побудував графік, що порівнює істинні та прогнозовані значення для перших 100 зразків тестової вибірки. На графіку видно, що помаранчева лінія (Прогнози ANFIS) дуже близько

слідую за синьою лінією (Істинні значення), коректно відтворюючи як загальний тренд, так і локальні коливання рівня ризику. Це візуально підтверджує високу якість моделі.



Висновки

У ході виконання лабораторної роботи було успішно реалізовано та навчено нейро-нечітку систему типу ANFIS для задачі прогнозування ризиків. Навчився виконувати ключові етапи роботи з нечіткими системами, як-от фазифікація та дефазифікація даних, а також реалізував повноцінну модель для прогнозування ризиків у виробничому процесі, використовуючи сучасні бібліотеки Python, такі як scikit-fuzzy та PyTorch. Фінальним етапом було проведення експериментальних досліджень для оцінки ефективності розробленої моделі.

Результати експериментів показали, що ANFIS-модель має значно вищу точність порівняно з неадаптивною FIS. Візуалізація прогнозів підтвердила, що модель здатна точно відстежувати зміни цільової змінної.

Це доводить, що гібридні нейро-нечіткі системи є ефективним інструментом для моделювання складних нелінійних процесів, оскільки вони поєднують здатність нейронних мереж до навчання на даних з інтерпретованістю нечіткої логіки.